hdoc day 30 by trishil chauhan

Aim

To implement a menu-driven C program for bubble sorting and binary searching, insertion and deletion at valid positions, and right rotation by k positions.

C Program

```
#include <stdio.h>
#define MAX 100

void display(int a[], int n) {
    if (n == 0) { printf("Array is empty.\n"); return; }
    printf("Array: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main(void) {
    int a[MAX], n, choice, key, pos, value, k;
    printf("Enter number of elements (0-%d): ", MAX);
    if (scanf("%d", &n) != 1 || n < 0 || n > MAX) {
        printf("Invalid array size.\n");
        return 1;
    }
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &a[i]) != 1) return 1;
    }

    do {
        printf("\n1. Bubble Sort and Binary Search\n");
        printf("2. Insert at position\n");
        printf("3. Delete at position\n");
```

```
        printf("4. Rotate right by k positions\n");
        printf("5. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) return 1;

        switch (choice) {
            case 1: {
                if (n == 0) { printf("Array is empty.\n"); break; }
                bubbleSort(a, n);
                printf("Sorted "); display(a, n);
                printf("Enter element to search: ");
                if (scanf("%d", &key) != 1) return 1;
                int index = binarySearch(a, n, key);
                if (index == -1) printf("Element not found.\n");
                else printf("Found at position %d.\n", index + 1);
                break;
            }
            case 2:
                if (n == MAX) { printf("Array is full.\n"); break; }
                printf("Enter position (1-%d) and value: ", n + 1);
                if (scanf("%d%d", &pos, &value) != 2) return 1;
                if (pos < 1 || pos > n + 1) {
                    printf("Invalid position.\n"); break;
                }
                for (int i = n; i >= pos; i--) a[i] = a[i - 1];
                a[pos - 1] = value;
                n++;
                display(a, n);
                break;
            case 3:
                if (n == 0) { printf("Array is empty.\n"); break; }
                printf("Enter position to delete (1-%d): ", n);
                if (scanf("%d", &pos) != 1) return 1;
                if (pos < 1 || pos > n) {
                    printf("Invalid position.\n"); break;
                }
                for (int i = pos - 1; i < n - 1; i++) a[i] = a[i + 1];
                n--;
                display(a, n);
                break;
            case 4:
                if (n == 0) { printf("Array is empty.\n"); break; }
                printf("Enter non-negative k: ");
                if (scanf("%d", &k) != 1) return 1;
                if (k < 0) { printf("Invalid k.\n"); break; }
                k %= n;
                for (int step = 0; step < k; step++) {
                    int last = a[n - 1];
                    for (int i = n - 1; i > 0; i--) a[i] = a[i - 1];
                    a[0] = last;
                }
                display(a, n);
                break;
            case 5: printf("Exiting program.\n"); break;
            default: printf("Invalid choice.\n");
        }
    } while (choice != 5);
    return 0;
}
```

Sample Run

```
Enter number of elements (0-100): 5
Enter 5 elements: 4 1 8 3 6
Enter choice: 1
```

```
Sorted Array: 1 3 4 6 8
Enter element to search: 6
Found at position 4.
Enter choice: 2
Enter position (1-6) and value: 3 9
Array: 1 3 9 4 6 8
Enter choice: 3
Enter position to delete (1-6): 2
Array: 1 9 4 6 8
Enter choice: 4
Enter non-negative k: 2
Array: 6 8 1 9 4
Enter choice: 5
Exiting program.
```

Result

The program performs all requested array operations using a menu. Positions are 1-based, and binary search is applied after sorting the array in ascending order.